Object Oriented Programming: Python BootCamp

HIT-emlyon BSc Program
April-May, Harbin | © Imène BRIGUI | Saeed VARASTEH

Lecture 09: Data Exploration with Pandas

The first step after reading in the data, is to take a look around, familiarize yourself and get acquainted with the data.

```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

```python
# Pandas Options
pd.set_option('display.max_columns', 15)
pd.set_option("display.max_rows", 15)
pd.set_option('display.float_format', lambda x: '%.3f' % x)
```

Loading Data into Pandas

Read Data

```python
data = pd.read_csv('data/house.csv')
print( data.shape )
```

```python
data.head(5)
```

```python
data.tail(5)
```

Data Cleaning

Drop Columns

Let's start by dropping columns, which we won't be using.

```python
data.columns
```

```python
data = data.drop(columns=["country","postcode"])
```

```python
print( data.shape )
data.head(5)
```

Dealing with NaNs:

```python
data.isna().sum()
```

```python
data.isnull().sum()
```

```python
data.dropna(inplace=True)
```

```python
print( data.shape )
data.head(5)
```

Rename Columns

```python
data.rename(columns={
        'rentEstimate_currentPrice': 'rentPrice',
        'saleEstimate_currentPrice': 'salePrice'
    },
    inplace=True
)
data.head(5)
```

```python
data.columns
```

Type Conversion

Notice anything off with the data types?

```python
data.dtypes
```

Lets fix column type:

```python
data['propertyType'] = data.propertyType.astype('category')
data.head(5)
```

Creating New Column

```python
data['above_mean'] = data.salePrice > data.salePrice.mean()
data.head(5)
```

Using `assign` function:

```python
data.assign(above_mean=lambda x: x.salePrice > x.salePrice.mean())
data.head(5)
```

lambda functions: These small, anonymous functions can receive multiple arguments, but can only contain one expression (the return value).

Filtering Data

```python
data.loc[(data['propertyType'] == 'Purpose Built Flat') & (data['bedrooms'] >=2.0) & (data['bathrooms'] <= 2.0)]
```

Conditional change

```python
data['above_mean'].value_counts()
```

```python
data.loc[ data['above_mean'] == False,'above_mean' ] = 'yes'
```

```python
data.loc[ data['above_mean'] == True,'above_mean' ] = 'no'
```

```python
data.head(5)
```

Sorting values

```python
data.sort_values(['salePrice','bathrooms'],ascending=[1,0])
```

Encodings

sklearn `LabelEncoder`

```python
data.tenure.value_counts()
```

```python
from sklearn.preprocessing import LabelEncoder
tenure_encoder = LabelEncoder()
data['tenure'] = tenure_encoder.fit_transform( data['tenure'] )
```

```python
data.head(5)
```

```python
data.tenure.value_counts()
```

```python
print( {k: v for k, v in enumerate(list(tenure_encoder.classes_))} )
```

pandas One Hot Encoder

```python
data_encoder = pd.get_dummies(data, columns=["tenure"])
print( data_encoder.shape )
data_encoder.head()
```

Discretization

Use the `pd.cut()` function to create values bins of equal width:

```python
data["salePriceBins"] = pd.cut(data["salePrice"], bins=3, labels=['low', 'medium', 'high'])
data.head(5)
```

```python
data["salePriceBins"].value_counts()
```

Data Exploration

Count total properties by number of bathrooms

```python
data["bathrooms"].unique()
```

```python
data["bathrooms"].value_counts()
```

```python
data.groupby("bathrooms").size()
```

```python
data.groupby("bathrooms")['salePrice'].mean()
```

Pivot Table

A pivot table is a table of grouped values that aggregates the individual items of a more extensive table within one or more discrete categories.

```python
data.head(5)
```

```python
data.pivot_table(index='bathrooms', columns='salePriceBins', values='salePrice', aggfunc='mean')
```

Crosstabs

The `pd.crosstab()` function provides an easy way to create a frequency table.

```python
pd.crosstab(index=data["bathrooms"],columns=data["salePriceBins"])
```

Describe

```python
data.describe()
```

Data Visualization

In this section, we will learn how to visualize data using pandas along with the Matplotlib and Seaborn libraries for additional features. We will create a variety of visualizations that will help us better understand our data.

Before everthing, to embed SVG-format plots in the notebook, we will also call the `%config` and `%matplotlib inline` magics:

```python
%config InlineBackend.figure_formats = ['svg']
%matplotlib inline
```

```python
data.head(5)
```

Plotting with Pandas

We can create a variety of visualizations using the `plot()` method.

Line Plot

The plot() method returns an Axes object that can be modified further (e.g., to add reference lines, annotations, labels, etc.).

```python
data.loc[100:110].plot(x="rentPrice", y="salePrice", title='salePrice 0:10', ylabel='salePrice', alpha=0.8)
plt.show()
```

Bar Plot

```python
plot_data = data.pivot_table(index='bathrooms', columns='salePriceBins', values='salePrice', aggfunc='mean')
plot_data
```

Pandas offers other plot types via the `kind` parameter, so we specify `kind='bar'` when calling the `plot()` method. Then, we further format the visualization using the `Axes` object returned by the `plot()` method:

```python
ax = plot_data.plot(
    kind='bar', rot=0, xlabel='', ylabel='Price Mean', fontsize=8,
    figsize=(12, 1.5), title='Number of bathrooms | Price Bins',
)
ax.legend(title='', loc='center', bbox_to_anchor=(0.5, -0.3), ncol=3, frameon=False)
plt.show()
```

EXERCISES 06 - PART ONE and TWO

HOMEWORK 05

SELF-STUDY

Summary:

Create Test Objects

Operator	Description
`pd.DataFrame(np.random.rand(20,5))`	5 columns and 20 rows of random floats
`pd.Series(my_list)`	Create a series from an iterable my_list
`df.index = pd.date_range('1900/1/30', periods=df.shape[0])`	Add a date index

Viewing/Inspecting Data

Operator	Description
`df.head(n)`	First n rows of the DataFrame
`df.tail(n)`	Last n rows of the DataFrame
`df.shape`	Number of rows and columns
`df.info()`	Index, Datatype and Memory information
`df.describe()`	Summary statistics for numerical columns
`s.value_counts(dropna=False)`	View unique values and counts
`df.apply(pd.Series.value_counts)`	Unique values and counts for all columns

Selection

Operator	Description
`df[col]`	Returns column with label col as Series
`df[[col1, col2]]`	Returns columns as a new DataFrame
`s.iloc[0]`	Selection by position
`s.loc['index_one']`	Selection by index
`df.iloc[0,:]`	First row
`df.iloc[0,0]`	First element of first column

Data Cleaning

Operator	Description
`df.columns = ['a','b','c']`	Rename columns
`pd.isnull()`	Checks for null Values, Returns Boolean Arrray
`pd.notnull()`	Opposite of pd.isnull()
`df.dropna()`	Drop all rows that contain null values
`df.dropna(axis=1)`	Drop all columns that contain null values
`df.dropna(axis=1,thresh=n)`	Drop all rows have have less than n non null values
`df.fillna(x)`	Replace all null values with x
`s.fillna(s.mean())`	Replace all null values with the mean
`s.astype(float)`	Convert the datatype of the series to float
`s.replace(1,'one')`	Replace all values equal to 1 with 'one'
`s.replace([2,3],['two', 'three'])`	Replace all 2 with 'two' and 3 with 'three'
`df.rename(columns=lambda x: x + 1)`	Mass renaming of columns
`df.rename(columns={'old_name': 'new_ name'})`	Selective renaming
`df.set_index('column_one')`	Change the index
`df.rename(index=lambda x: x + 1)`	Mass renaming of index

Filter, Sort, and Groupby

Operator	Description
`df[df[col] > 0.6]`	Rows where the column col is greater than 0.6
`df[(df[col] > 0.6) & (df[col] < 0.8)]`	Rows where 0.8 > col > 0.6
`df.sort_values(col1)`	Sort values by col1 in ascending order
`df.sort_values(col2,ascending=False)`	Sort values by col2 in descending order.5
`df.sort_values([col1,col2],ascending=[True,False])`	Sort values by col1 in ascending order then col2 in descending order
`df.groupby(col)`	Returns a groupby object for values from one column
`df.groupby([col1,col2])`	Returns groupby object for values from multiple columns
`df.groupby(col1)[col2]`	Returns the mean of the values in col2, grouped by the values in col1
`df.pivot_table(index=col1,values=[col2,col3],aggfunc=mean)`	Create a pivot table that groups by col1 and calculates the mean of col2 and col3
`df.groupby(col1).agg(np.mean)`	Find the average across all columns for every unique col1 group
`df.apply(np.mean)`	Apply the function np.mean() across each column
`nf.apply(np.max,axis=1)`	Apply the function np.max() across each row

Join/Combine

Operator	Description
`df1.append(df2)`	Add the rows in df1 to the end of df2 (columns should be identical)
`pd.concat([df1, df2],axis=1)`	Add the columns in df1 to the end of df2 (rows should be identical)
`df1.join(df2,on=col1, how='inner')`	SQL-style join the columns in df1 with the columns on df2 where the rows for col have identical values. The 'how' can be 'left', 'right', 'outer' or 'inner'

Statistics

Operator	Description
`df.describe()`	Summary statistics for numerical columns
`df.mean()`	Returns the mean of all columns
`df.corr()`	Returns the correlation between columns in a DataFrame
`df.count()`	Returns the number of non-null values in each DataFrame column
`df.max()`	Returns the highest value in each column
`df.min()`	Returns the lowest value in each column
`df.median()`	Returns the median of each column
`df.std()`	Returns the standard deviation of each column

Importing Data

Operator	Description
`pd.read_csv(filename)`	From a CSV file
`pd.read_table(filename)`	From a delimited text file (like TSV)
`pd.read_excel(filename)`	From an Excel file
`pd.read_sql(query, connection_object)`	Read from a SQL table/database
`pd.read_json(json_string)`	Read from a JSON formatted string, URL or file.
`pd.read_html(url)`	Parses an html URL, string or file and extracts tables to a list of dataframes
`pd.read_clipboard()`	Takes the contents of your clipboard and passes it to read_table()
`pd.DataFrame(dict)`	From a dict, keys for columns names, values for data as lists

Exporting Data

Operator	Description
`df.to_csv(filename)`	Write to a CSV file
`df.to_excel(filename)`	Write to an Excel file
`df.to_sql(table_name, connection_object)`	Write to a SQL table
`df.to_json(filename)`	Write to a file in JSON format
